Programming Assignment 2

- **Purpose of Homework:**
  - Algorithm design and analysis, like any skill, can only be developed through consistent practice and feedback. Whether it's cooking, playing basketball, integration, gardening, interviewing, or teaching, theoretical knowledge alone is not sufficient. The comfortable feeling of "Oh, sure, I get it" after following a well-presented lecture or hearing a TA explain a homework solution is a seductive, yet dangerous trap. True understanding comes from doing the thing—by actually solving the problems yourself.
  - The homework assignments are your opportunity to practice. Lectures, textbooks, office hours, labs, and guided problem sets are designed to build intuition and provide justification for the skills we want you to develop. However, the most effective way to develop those skills is by attempting to solve the problems on your own. The process is far more important than the final solution.
  - Expect to get stuck. It's normal to have no idea where to start on some problems. That's why you have access to a textbook, lecture slides, and discussions. The journey of wrestling with the problem is an essential part of the learning process.

Homework Guidelines

- **Collaboration and Academic Integrity:**
  - You are encouraged to work together on homework problems, but you must list everyone you worked with for each problem.
  - You must code everything based on your interpretation and properly cite every external source you use, including ideas from other students. The only sources that you are not required to cite are the official course materials (lectures, notes, homework solutions).
  - Plagiarism is strictly prohibited. Using ideas from other sources or people without citation is considered plagiarism. Copying verbatim from any source, even with citation or permission, is also considered plagiarism. Don't cheat.
- **Programming Guidelines:**
  - You are encouraged, but not required, to write clean and descriptive code. This helps with debugging, especially if you want to ask for help at office hours.
  - All solutions must be coded from scratch, i.e. we will **not** provide boilerplate code for any programming problem. One implication is that you must write your own code to read the inputs and print the outputs based on our convention.
  - Pay close attention to the instructions for each problem. Your code might fail some test cases due to non-algorithimic errors, e.g. integer overflow, runtime errors, stack overflow, etc.

Policy on Academic Integrity and AI Usage

- **LLM Usage:** To ensure the development of independent algorithmic thinking, the use of Large Language Models (e.g., ChatGPT, Gemini) for generating assignment solutions is strictly discouraged.
- **Oral Verification:** At the conclusion of the semester, the instructor reserves the right to perform *randomized oral examinations* on submitted work.
  - Students must be able to provide a precise, step-by-step explanation of their logic and the underlying mechanics of their algorithm.
  - Failure to demonstrate a thorough understanding of the submitted work during this oral defense will result in a rejection of the assignment.

Note: Authenticity in your reasoning is the primary criterion for evaluation.

Programming Assignment #2 Guidelines

Points

- Programming Assignment #2 is worth up to **500 points**.
- Students enrolled in the **Honors** option are required to solve all programming problems.
- **Honors points count for everyone:** Everyone else is welcome to attempt them as well. If you solve any Honors problems, the points you earn are added to your total homework points in the same way as the rest of the assignment.

Overview

Problem	File Name	Time Limit	Points
Look MoM, No Quadratic!	`lookmom.*`	2 seconds	50
Parkour Master	`parkour.*`	2 seconds	50
Zombie Apocalypse	`apocalypse.*`	2 seconds	60
Enchanted Runes	`runes.*`	2 seconds	60
Galactic Mining	`mining.*`	2 seconds	60
Celestial Forging	`forging.*`	2 seconds	60
Wacky Solitaire	`solitaire.*`	2 seconds	80
Card Wars	`cardwars.*`	2 seconds	80
			Total: 500

The `.*` corresponds to the file extension of the programming language of your choice. Note that the time limits above are the **base rate** only, the actual time limit will be multiplied with the factors below depending on the language.

Language	C++	C	Java	Python[1]	Rust
Extension	`.cpp`	`.c`	`.java`	`.py`	`.rs`
Time Multiplier	1x	1x	2x	4x	1x

For example, if you submit a Python solution for Look MoM, No Quadratic!, your program must run within 4×2 second $= 8$ seconds for every test case.

Submission Instructions:

- Submit your solutions on Gradescope, one file per problem. Each submitted file **must** follow this format: `[problem name].*`. Failure to adhere to this format will result in your solution not being graded.
- The following languages are supported: **C++, C, Java, Python, Rust**. However, you are **NOT** restricted to these languages only; let us know your favorite language and we will create a grading pipeline for it :)
- For each language, you are only allowed to use standard libraries as part of your code. Usage of external libraries (e.g. `numpy`, `sortedcontainers` in Python) will result in a **Compilation Error**.

[1] The autograder runs your code on PyPy, which is typically much faster than regular Python.

1 Look MoM, No Quadratic!

Bingo is a young girl with a profound passion in mathematics and computer science. Recently, she came across the `QuickSelect` algorithm and became fascinated with the median-of-medians approach for linear-time selection. The median-of-medians approach works by partitioning an array of integers into smaller subarrays and computing the median in each subarray, then finds an approximate pivot by taking the median of all such medians. The approximate pivot guarantees `QuickSelect` run in linear time even in the worst case scenario, instead of the naive quadratic time (look mom!).

Being curious, Bingo starts to wonder why this median-of-median approach works well and wants to experiment it on a two-dimensional array. In particular, she draws some numbers to create an $m \times n$ matrix and wants to find the best dimensions to capture the median across all submatrices. Unfortunately, Bingo stumbles upon two major problems: firstly, she has no idea what the dimensions should be, and secondly, she realizes implementing the median-of-median approach is more difficult than expected.

Bingo does not want to just give up however, so she brainstorms a simpler variant and asks you for help! Instead of finding the median-of-medians, she wants to experiment with her new **maximum**-of-medians approach: can you help her find the largest median across all $w \times h$ submatrices?

There are many definitions for the median, so Bingo uses her favorite one: the median of a list of integers is the smallest integer x such that at least half of the elements is greater than or equal to x and at least half of the elements is less than or equal to x.

Input

The first line of input contains four integers, m, n, w, h, the dimensions of the matrix and the submatrix, respectively.

m lines of input follow. The ith line contains n integers $a_{i,1}, \ldots, a_{i,n}$.

Output

Output the maximum of medians across all $w \times h$ submatrices.

Constraints

- $1 \leq m, n \leq 2000$
- $1 \leq w \leq m, 1 \leq h \leq n$
- $|a_i| \leq 10^9$

Examples

Sample Input	Sample Output
3 3 2 2 4 8 5 2 3 7 1 6 9	6

Explanation

In the sample test case, the medians of all 2×2 regions are $3, 5, 2, 6$.

2 Parkour Master

Pucheng is a parkour master. He enjoys jumping from rooftop to rooftop, challenging himself to perform as many jumps as possible. The UW-Madison campus forms a line of n buildings, where the ith building has a height of h_i. Pucheng would like to plan a parkour route on the roofs these buildings. However, he cannot defy the laws of physics. He can only jump from building i to building j if every building in between the two is strictly shorter than building i. This includes building j; we require $h_i > h_j$.

Pucheng can start and end his parkour route anywhere on campus. Help him determine the maximum number of rooftops he can visited. Note that the number of rooftops he visits is exactly one more than the number of jumps he makes.

Input

The first line has an integer n, the number of buildings. The next line has n integers $h_1, h_2, \cdots, h_n$ that describe the heights of the buildings.

Output

Print one integer, namely the maximum number of rooftops that Pucheng can visit.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $1 \leq h_i \leq 10^9$

Examples

Sample Input	Sample Output
10 20 15 17 35 25 40 12 19 13 12	5

3 Zombie Apocalypse

Oh no! There has been a T-virus outbreak in Racoon City! A group citizens find themselves stuck in the Racoon City police department and are running short on supplies. Leon Kennedy realizes he is the only person brave enough to go to City Hall to gather supplies. After crossing the perilous environment, he finds a collection of n supply boxes, where each box i weighs w_i. Leon knows that he can only manage a total weight of k if he wants to get back safely, so he'll have to make multiple trips. What is the smallest number of trips he needs to make to get all the supplies back?

Input

The first line has two integers n and k, where n is the number of supply boxes and k is the maximum weight he can carry. The next line has n integers $w_1, w_2, \cdots, w_n$ that describe the weights of the supply boxes.

Output

Print one integer, namely the smallest number of trips Leon needs to make.

Constraints

- $1 \leq n \leq 20$
- $1 \leq x \leq 10^9$
- $1 \leq w_i \leq x$

Examples

Sample Input	Sample Output
4 10 4 8 6 1	2

4 Enchanted Runes

The Grand Mage of the Arcane Order is preparing a powerful spell using a sequence of ancient magical runes. The spell is written as a perfectly balanced sequence of opening `(` and closing `)` runes. Just like a correct mathematical expression, each opening rune corresponds to a specific matching closing rune.

To activate the spell, the Mage must imbue the runes with elemental magic. Each rune can either be left uncharged, imbued with Light magic, or imbued with Dark magic.

However, arcane energy is highly volatile. To maintain the stability of the spell, the magical imbuement must strictly follow three rules:

1. Each rune is either left uncharged, imbued with Light magic, or imbued with Dark magic.
2. For any pair of matching (linked) runes, exactly one of them must be imbued with magic. The other must remain uncharged.
3. No two adjacent runes in the sequence can be imbued with the same type of magic (i.e., Light cannot be immediately next to Light, and Dark cannot be immediately next to Dark). Uncharged runes do not carry any elemental magic.

The Grand Mage needs your help to find the total number of different ways to safely imbue the rune sequence. Two ways of imbuement are considered different if they differ in the magic state (uncharged, Light, or Dark) of at least one rune.

Input

The first and only line contains a single string s ($2 \leq |s| \leq 700$), which represents the perfectly balanced sequence of runes. It is guaranteed that s forms a correct bracket sequence.

Output

Output one integer, the number of valid ways to imbue the rune sequence. Since the result can be large, please output it modulo $10^9 + 7$.

Constraints

- $2 \leq |s| \leq 700$
- s consists only of the characters `(` and `)`.

Examples

Sample Input 1	Sample Output 1
`(())`	12

Sample Input 2	Sample Output 2
`(()())`	40

Sample Input 3	Sample Output 3
`()`	4

5 Galactic Mining

Captain Nova is leading an expedition in the resource-rich Orion Asteroid Belt. The belt consists of a linear sequence of n asteroid sectors, where each sector i contains p_i tons of valuable stardust.

To extract the stardust, Nova has a fleet of k autonomous mining drones. Each drone is strictly programmed to harvest from exactly m consecutive sectors. To avoid collision and interference, no two drones can overlap in the sectors they harvest. In other words, Nova must choose k non-overlapping continuous segments of length m from the n sectors.

Your task is to help Captain Nova determine the maximum total amount of stardust that can be extracted by strategically deploying all k drones.

Input

The first line contains three integers n, m, and k ($1 \leq m \times k \leq n \leq 5000$) — the number of asteroid sectors, the number of consecutive sectors each drone mines, and the total number of drones, respectively.

The second line contains n integers $p_1, p_2, \ldots, p_n$ ($0 \leq p_i \leq 10^9$) — the amount of stardust in each sector.

Output

Print a single integer representing the maximum possible tons of stardust the drones can extract.

Constraints

- $1 \leq m \times k \leq n \leq 5000$
- $0 \leq p_i \leq 10^9$

Examples

Sample Input 1	Sample Output 1
5 2 1 1 2 3 4 5	9

Sample Input 2	Sample Output 2
7 1 3 2 10 7 18 5 33 0	61

6 Celestial Forging

The Master Artificer of the Cosmos is forging a legendary staff. The staff's core must be constructed by lining up exactly n_1 Solar crystals and n_2 Lunar crystals in a single row.

To maintain the staff's elemental balance, an arrangement is considered stable only if there are never strictly more than k_1 Solar crystals placed successively, and never strictly more than k_2 Lunar crystals placed successively.

All $n_1 + n_2$ crystals must be present in the final staff. All Solar crystals are considered completely indistinguishable from one another, and similarly, all Lunar crystals are considered indistinguishable from one another.

Your task is to find the total number of stable ways to arrange the crystals. However, the number of such ways can be large, so the Master is interested in only the last **eight** digits of your answer.

Input

The only line of input contains four space-separated integers n_1, n_2, k_1, and k_2, representing the number of Solar crystals, the number of Lunar crystals, the maximum acceptable number of consecutive Solar crystals, and the maximum acceptable number of consecutive Lunar crystals, respectively.

Output

Output eight digits, representing the last eight digits of the number of stable arrangements of the crystals.

Constraints

- $1 \leq n_1, n_2 \leq 100$
- $1 \leq k_1, k_2 \leq 10$

Examples

Sample Input 1	Sample Output 1
2 1 1 10	0000001

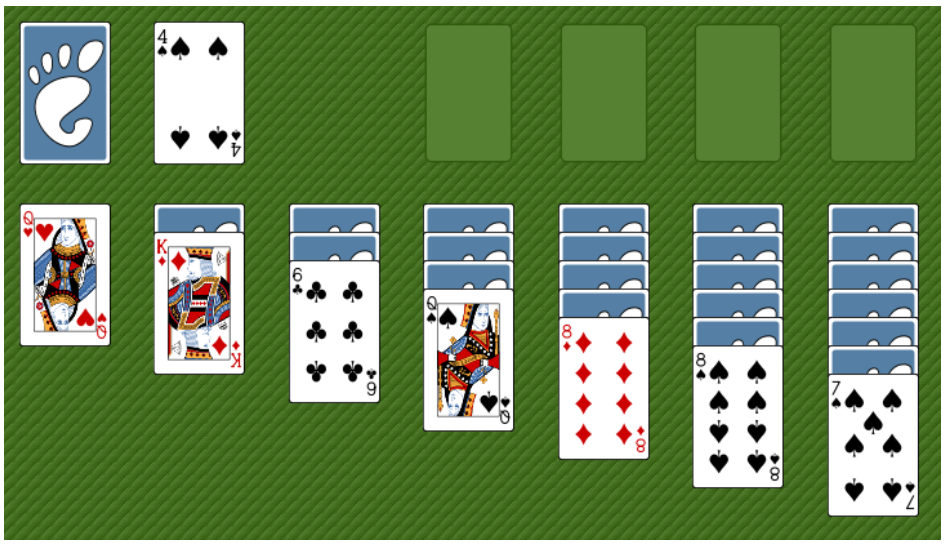
Sample Input 2	Sample Output 2
2 3 1 2	00000005

Sample Input 3	Sample Output 3
2 4 1 1	00000000

7 Wacky Solitaire

After an exhausting exam week, Brandon is fed up with schoolwork and decides to play some games to blow off steam. One of the games he has been playing is *"Wacky Solitaire"*, which follows similar rules as Klondike where the player aims to sort all cards in the correct order. However, the game has some key differences:

- The cards all share the same suit and only differ by its value, which is always a positive integer.
- The game's AI is *adversarial*: since the player can move cards between different piles but not those from the same pile, it always divides the decks into the smallest number of piles such that cards in the same pile are always in **strictly** decreasing order of value.



Brandon is not a good solitaire player, so he often gets frustrated with losing the game. However, he is a good hacker, and he just found a loophole in the game's program that allows him to modify the deck to make the game easier! Brandon can choose a contiguous selection of cards and increase or decrease all of their values by an integer x. Unfortunately, his tools are rather limited, so he can only update them by at most a threshold, i.e. $|x| \leq k$.

Obviously, Brandon would like to modify the cards so that the AI can deal him as many piles as possible to make the game easier. Can you help Brandon find the optimal update?

Input

The first line of input contains two integers, n and k, the number of cards in the deck and the maximum threshold Brandon can modify his cards.

The second line of input contains n positive integers, $a_1, \ldots, a_n$, the card values.

Output

Output the number of piles the AI will deal if Brandon modifies the cards optimally.

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $0 \leq k \leq 10^9$
- $1 \leq a_i \leq 10^9$

Examples

Sample Input	Sample Output
6 5 4 2 6 3 8 4	4

Explanation

Brandon can choose to increase every card between $\ell = 2$ to $r = 5$ by $x = 3$, then the new card values will be $[\underline{\mathbf{4}}, \underline{\mathbf{5}}, 9, \underline{\mathbf{6}}, \underline{\mathbf{11}}, 4]$. With this deck, the AI will deal him four piles: $[4], [5, 4], [9, 6], [11]$.

8 Card Wars

Jake has just bought a couple new decks of playing cards and invites Finn to play a game with him! Jake's decks belong to a common style in Ooo that is similar to French-suited playing cards: each card assumes a unique pair of rank (labeled from 1 to n) and suit (labeled from 1 to k).

Jake wants to play a simple game with Finn. Each of them receives and shuffles their own deck, then takes turn drawing their topmost cards. They earn a **synergy** if Finn's card and Jake's card have the same rank; otherwise, either Finn or Jake needs to discard their current card and draws a new one from their deck. The game continues until they both run out of cards.

After several rounds, BMO walks in and asks if Finn and Jake are going to prepare dinner soon. Jake invites BMO to watch them play another game; after the round concludes, BMO claims: *"I know you guys have so much fun when uncovering synergies, but I can guide you towards a strategy that helps you two get the highest synergy."*

Finn and Jake are surprised, but start to get curious. Can you help them find out how BMO can calculate their best synergy for each game?

Input

The first line of input contains two integers, n and k, the number of ranks and suits, respectively.

The second line of input contains $n \times k$ integers $a_1, \ldots, a_{nk}$, denoting the ranks of Finn's deck. It is guaranteed that each rank appears exactly k times in Finn's deck.

The third line of input contains $n \times k$ integers $b_1, \ldots, b_{nk}$, denoting the ranks of Jake's deck. It is also guaranteed that each rank appears exactly k times in Jake's deck.

Output

Output the highest synergy Finn and Jake can obtain.

Constraints

- $1 \leq n \leq 10^4$
- $1 \leq k \leq 20$

Examples

Sample Input	Sample Output
3 2 3 1 2 3 1 2 2 1 3 1 3 2	4